

04_exercise

September 28, 2025

```
[27]: from pathlib import Path
import numpy as np
import torch
from typing import List
from torch.nn.utils.rnn import pad_sequence
from mltrainer import rnn_models, Trainer
from torch import optim

from mads_datasets import datatools
import mltrainer
mltrainer.__version__
```

```
[27]: '0.2.4'
```

1 1 Iterators

We will be using an interesting dataset. [link](#)

From the site: > The SmartWatch Gestures Dataset has been collected to evaluate several gesture recognition algorithms for interacting with mobile applications using arm gestures. Eight different users performed twenty repetitions of twenty different gestures, for a total of 3200 sequences. Each sequence contains acceleration data from the 3-axis accelerometer of a first generation Sony SmartWatch™, as well as timestamps from the different clock sources available on an Android device. The smartwatch was worn on the user's right wrist.

```
[28]: from mads_datasets import DatasetFactoryProvider, DatasetType
from mltrainer.preprocessors import PaddedPreprocessor
preprocessor = PaddedPreprocessor()

gesturesdatasetfactory = DatasetFactoryProvider.create_factory(DatasetType.
↳GESTURES)
streamers = gesturesdatasetfactory.create_datastreamer(batchsize=32,↳
↳preprocessor=preprocessor)
train = streamers["train"]
valid = streamers["valid"]
```

2025-09-28 15:41:11.876 | INFO |

```
mads_datasets.base:download_data:121 - Folder
```

```
already exists at /Users/nickreinders/.cache/mads_datasets/gestures
```

```
100%|      | 2600/2600 [00:00<00:00, 8989.64it/s]
```

```
100%|      | 651/651 [00:00<00:00, 12982.07it/s]
```

```
[29]: len(train), len(valid)
```

```
[29]: (81, 20)
```

```
[30]: trainstreamer = train.stream()
      validstreamer = valid.stream()
      x, y = next(iter(trainstreamer))
      x.shape, y
```

```
[30]: (torch.Size([32, 29, 3]),
      tensor([ 3, 12, 11,  8,  5, 10, 19, 13,  2,  8,  5,  9,  4,  1, 18,  0,  8, 13,
               2,  9,  8,  3, 15,  7, 16,  4,  5,  2,  8, 15, 12,  1]))
```

Can you make sense of the shape? What does it mean that the shapes are sometimes (32, 27, 3), but a second time might look like (32, 30, 3)? In other words, the second (or first, if you insist on starting at 0) dimension changes. Why is that? How does the model handle this? Do you think this is already padded, or still has to be padded?

2 2 Excercises

Lets test a basemodel, and try to improve upon that.

Fill the gestures.gin file with relevant settings for `input_size`, `hidden_size`, `num_layers` and `horizon` (which, in our case, will be the number of classes...)

As a rule of thumbs: start lower than you expect to need!

```
[31]: from mltrainer import TrainerSettings, ReportTypes
      from mltrainer.metrics import Accuracy

      accuracy = Accuracy()
```

```
[32]: model = rnn_models.BaseRNN(
      input_size=3,
      hidden_size=64,
      num_layers=1,
      horizon=20,
      )
```

Test the model. What is the output shape you need? Remember, we are doing classification!

```
[33]: yhat = model(x)
      yhat.shape
```

```
[33]: torch.Size([32, 20])
```

Test the accuracy

```
[34]: accuracy(y, yhat)
```

```
[34]: 0.0625
```

What do you think of the accuracy? What would you expect from blind guessing?

Check shape of y and yhat

```
[35]: yhat.shape, y.shape
```

```
[35]: (torch.Size([32, 20]), torch.Size([32]))
```

And look at the output of yhat

```
[36]: yhat[0]
```

```
[36]: tensor([-0.1327,  0.1308,  0.0893,  0.0554,  0.0623, -0.0512,  0.0406,  0.0355,
          -0.1240, -0.1806,  0.1472,  0.0291, -0.0262,  0.0911, -0.0255,  0.1806,
          -0.0577,  0.0255, -0.0778,  0.1101], grad_fn=<SelectBackward0>)
```

Does this make sense to you? If you are unclear, go back to the classification problem with the MNIST, where we had 10 classes.

We have a classification problem, so we need Cross Entropy Loss. Remember, [this has a softmax built in](#)

```
[37]: loss_fn = torch.nn.CrossEntropyLoss()
      loss = loss_fn(yhat, y)
      loss
```

```
[37]: tensor(3.0034, grad_fn=<NllLossBackward0>)
```

```
[38]: import torch
      if torch.backends.mps.is_available() and torch.backends.mps.is_built():
          device = torch.device("mps")
          print("Using MPS")
      elif torch.cuda.is_available():
          device = "cuda:0"
          print("using cuda")
      else:
          device = "cpu"
          print("using cpu")

      # on my mac, at least for the BaseRNN model, mps does not speed up training
      # probably because the overhead of copying the data to the GPU is too high
      # so i override the device to cpu
      device = "cpu"
```

```
# however, it might speed up training for larger models, with more parameters
```

Using MPS

Set up the settings for the trainer and the different types of logging you want

```
[39]: settings = TrainerSettings(  
    epochs=3, # increase this to about 100 for training  
    metrics=[accuracy],  
    logdir=Path("gestures"),  
    train_steps=len(train),  
    valid_steps=len(valid),  
    reporttypes=[ReportTypes.TOML, ReportTypes.TENSORBOARD, ReportTypes.MLFLOW],  
    scheduler_kwargs={"factor": 0.5, "patience": 5},  
    earlystop_kwargs = {  
        "save": False, # save every best model, and restore the best one  
        "verbose": True,  
        "patience": 5, # number of epochs with no improvement after which  
        ↪ training will be stopped  
        "delta": 0.0, # minimum change to be considered an improvement  
    }  
)  
settings
```

```
[39]: epochs: 3  
metrics: [Accuracy]  
logdir: gestures  
train_steps: 81  
valid_steps: 20  
reporttypes: [<ReportTypes.TOML: 'TOML'>, <ReportTypes.TENSORBOARD:  
'TENSORBOARD'>, <ReportTypes.MLFLOW: 'MLFLOW'>]  
optimizer_kwargs: {'lr': 0.001, 'weight_decay': 1e-05}  
scheduler_kwargs: {'factor': 0.5, 'patience': 5}  
earlystop_kwargs: {'save': False, 'verbose': True, 'patience': 5, 'delta': 0.0}
```

```
[40]: import torch.nn as nn  
import torch  
from torch import Tensor  
from dataclasses import dataclass  
  
@dataclass  
class ModelConfig:  
    input_size: int  
    hidden_size: int  
    num_layers: int  
    output_size: int  
    dropout: float = 0.0
```

```

class GRUModel(nn.Module):
    def __init__(
        self,
        config,
    ) -> None:
        super().__init__()
        self.config = config
        self.rnn = nn.GRU(
            input_size=config.input_size,
            hidden_size=config.hidden_size,
            dropout=config.dropout,
            batch_first=True,
            num_layers=config.num_layers,
        )
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: Tensor) -> Tensor:
        x, _ = self.rnn(x)
        last_step = x[:, -1, :]
        yhat = self.linear(last_step)
        return yhat

```

```

[41]: config = ModelConfig(
    input_size=3,
    hidden_size=64,
    num_layers=1,
    output_size=20,
    dropout=0.0,
)

```

```

[42]: import mlflow
from datetime import datetime

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("gestures")
model_dir = Path("gestures").resolve()
if not model_dir.exists():
    model_dir.mkdir(parents=True)

with mlflow.start_run():
    mlflow.set_tag("model", "modelname-here")
    mlflow.set_tag("dev", "your-name-here")
    config = ModelConfig(
        input_size=3,
        hidden_size=64,
        num_layers=1,
        output_size=20,
    )

```

```

        dropout=0.1,
    )

    model = GRUModel(
        config=config,
    )

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindata_loader=trainstreamer,
        validdataloader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()

    if not settings.earlystop_kwargs["save"]:
        tag = datetime.now().strftime("%Y%m%d-%H%M-")
        modelpath = model_dir / (tag + "model.pt")
        torch.save(model, modelpath)

```

/Users/nickreinders/Documents/Master Applied Data Science deeltijd
 HAN/Deployment/Raoul_repo/MADS-MachineLearning-course/.venv/lib/python3.11/site-
 packages/torch/nn/modules/rnn.py:123: UserWarning: dropout option adds dropout
 after all but last recurrent layer, so non-zero dropout expects num_layers
 greater than 1, but got dropout=0.1 and num_layers=1

```

warnings.warn(
2025-09-28 15:41:12.467 | INFO      |
mltrainer.trainer:dir_add_timestamp:24 - Logging

```

```

to gestures/20250928-154112
2025-09-28 15:41:12.468 | INFO      |
mltrainer.trainer:__init__:68 - Found

```

```

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
0%|          | 0/3 [00:00<?, ?it/s]2025-09-28

```

```

15:41:12.468 | INFO      |
mltrainer.trainer:__init__:68 - Found

```

```

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
100%|         | 81/81 [00:00<00:00, 313.48it/s]

```

```

2025-09-28 15:41:12.761 | INFO      |
mltrainer.trainer:report:209 - Epoch 0 train

```

```

2.8448 test 2.4210 metric ['0.1469']
100%|         | 81/81 [00:00<00:00, 332.87it/s]

```

```

2025-09-28 15:41:13.029 | INFO      |
mltrainer.trainer:report:209 - Epoch 1 train

2.2502 test 2.1143 metric ['0.2188']
100%|      | 81/81 [00:00<00:00, 337.02it/s]
2025-09-28 15:41:13.294 | INFO      |
mltrainer.trainer:report:209 - Epoch 2 train

2.1104 test 2.0335 metric ['0.2562']
100%|      | 3/3 [00:00<00:00, 3.66it/s]

```

Try to update the code above by changing the hyperparameters.

To discern between the changes, also modify the tag `mlflow.set_tag("model", "new-tag-here")` where you add a new tag of your choice. This way you can keep the models apart.

```

[43]: trainer.loop() # if you want to pick up training, loop will continue from the
      ↪ last epoch

```

```

100%|      | 81/81 [00:00<00:00, 333.37it/s]
2025-09-28 15:41:13.565 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 3
2025-09-28 15:41:13.573 | INFO      |
mltrainer.trainer:report:209 - Epoch 3 train

1.9920 test 1.9040 metric ['0.3641']
100%|      | 81/81 [00:00<00:00, 331.09it/s]
2025-09-28 15:41:13.843 | INFO      |
mltrainer.trainer:report:209 - Epoch 4 train

1.7970 test 1.6312 metric ['0.4859']
100%|      | 81/81 [00:00<00:00, 341.08it/s]
2025-09-28 15:41:14.107 | INFO      |
mltrainer.trainer:report:209 - Epoch 5 train

1.5684 test 1.4378 metric ['0.5391']
100%|      | 3/3 [00:00<00:00, 3.73it/s]

```

```

[44]: mlflow.end_run()

```

Exercises:

- try to improve the RNN model
- test different things. What works? What does not?
- experiment with either GRU or LSTM layers, create your own models. Have a look at `mltrainer.rnn_models` for inspiration.
- experiment with adding Conv1D layers. Think about the necessary input-output dimensions of your tensors before and after each layer.

You should be able to get above 90% accuracy with the dataset. Create a report of 1 a4 about your experiments.

vanaf hier dingen proberen

```
[45]: class LSTMmodel(nn.Module):
    def __init__(self, config) -> None:
        super().__init__()
        self.config = config
        self.rnn = nn.LSTM(
            input_size=config.input_size,
            hidden_size=config.hidden_size,
            dropout=config.dropout,
            batch_first=True,
            num_layers=config.num_layers,
        )
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x, _ = self.rnn(x)
        last_step = x[:, -1, :]      # neem laatste timestep
        yhat = self.linear(last_step)
        return yhat
```

```
[46]: config = ModelConfig(
    input_size=3,      # 3 sensorkanalen (accelerometer)
    hidden_size=128,   # groter dan 64 proberen
    num_layers=2,      # meer lagen
    output_size=20,    # 20 klassen (gestures)
    dropout=0.2,       # beetje regularisatie
)
```

```
[47]: with mlflow.start_run():
    mlflow.set_tag("model", "LSTM-128x2")
    mlflow.set_tag("dev", "nick")

    model = LSTMmodel(config=config)

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindataloader=trainstreamer,
        validdataloader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()
```

2025-09-28 15:52:04.480 | INFO |


```

mltrainer.trainer:dir_add_timestamp:24 - Logging
to gestures/20250928-155204
2025-09-28 15:52:04.481 | INFO      |
mltrainer.trainer:__init__:68 - Found

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
0%|          | 0/3 [00:00<?, ?it/s]2025-09-28
15:52:04.481 | INFO      |
mltrainer.trainer:__init__:68 - Found

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
100%|         | 81/81 [00:01<00:00, 76.21it/s]
2025-09-28 15:52:05.656 | INFO      |
mltrainer.trainer:report:209 - Epoch 0 train

2.6832 test 2.3405 metric ['0.1234']
100%|         | 81/81 [00:01<00:00, 80.52it/s]
2025-09-28 15:52:06.768 | INFO      |
mltrainer.trainer:report:209 - Epoch 1 train

2.2196 test 1.9807 metric ['0.3063']
100%|         | 81/81 [00:01<00:00, 75.13it/s]
2025-09-28 15:52:07.965 | INFO      |
mltrainer.trainer:report:209 - Epoch 2 train

1.8822 test 1.6257 metric ['0.3734']
100%|         | 3/3 [00:03<00:00, 1.16s/it]

```

```
[53]: trainer.loop()
```

```
mlflow.end_run()
```

```

100%|         | 81/81 [00:01<00:00, 73.31it/s]
2025-09-28 16:01:38.381 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 6
2025-09-28 16:01:38.389 | INFO      |
mltrainer.trainer:report:209 - Epoch 6 train

0.9211 test 0.8099 metric ['0.6547']
100%|         | 81/81 [00:00<00:00, 81.58it/s]
2025-09-28 16:01:39.485 | INFO      |
mltrainer.trainer:report:209 - Epoch 7 train

0.7690 test 0.6200 metric ['0.7500']
100%|         | 81/81 [00:01<00:00, 78.95it/s]
2025-09-28 16:01:40.621 | INFO      |

```

```
mltrainer.trainer:report:209 - Epoch 8 train
```

```
0.6217 test 0.6504 metric ['0.7516']
```

```
2025-09-28 16:01:40.621 | INFO      |
```

```
mltrainer.trainer:__call__:252 - best loss:
```

```
0.6200, current loss 0.6504.Counter 1/5.
```

```
100%|      | 3/3 [00:03<00:00,  1.15s/it]
```

we gaan hier conv1d toevoegen voor rnn

```
[49]: @dataclass
class HybridConfig:
    input_size: int          # aantal features (hier: 3 accelerometer-kanalen)
    hidden_size: int         # hidden size van RNN
    num_layers: int          # aantal RNN-lagen
    output_size: int         # aantal klassen (20 gestures)
    conv_channels: int = 16   # filters van Conv1D
    kernel_size: int = 3     # filtergrootte van Conv1D
    dropout: float = 0.0     # dropout voor regularisatie

[50]: class ConvLSTMmodel(nn.Module):
    def __init__(self, config: HybridConfig) -> None:
        super().__init__()
        self.config = config

        # Conv1D laag: zoekt korte patronen
        self.conv1 = nn.Conv1d(
            in_channels=config.input_size,          # input features (3
            ↪sensorkanalen)
            out_channels=config.conv_channels,      # aantal filters
            kernel_size=config.kernel_size
        )
        self.relu = nn.ReLU()

        # RNN laag (LSTM in dit voorbeeld)
        self.rnn = nn.LSTM(
            input_size=config.conv_channels,        # output van Conv1D = features
            ↪voor RNN
            hidden_size=config.hidden_size,
            num_layers=config.num_layers,
            dropout=config.dropout,
            batch_first=True,
        )

        # Lineaire laag naar klassen
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```

# x: (batch, timesteps, features) → (B, T, F)
# Conv1D verwacht (batch, channels, timesteps)
x = x.permute(0, 2, 1)  # (B, F, T)

x = self.conv1(x)        # (B, conv_channels, T_new)
x = self.relu(x)

# terug naar (B, T_new, conv_channels) voor RNN
x = x.permute(0, 2, 1)

x, _ = self.rnn(x)       # (B, T_new, hidden_size)
last_step = x[:, -1, :]  # laatste timestep
yhat = self.linear(last_step)  # (B, output_size)
return yhat

```

```

[51]: config = HybridConfig(
    input_size=3,      # accelerometer x,y,z
    hidden_size=128,   # groter dan 64
    num_layers=1,      # begin met 1 LSTM-laag
    output_size=20,    # 20 gestures
    conv_channels=16,  # aantal conv filters
    kernel_size=3,     # kernelgrootte
    dropout=0.2,       # beetje dropout
)

```

```

[54]: with mlflow.start_run():
    mlflow.set_tag("model", "Conv1D-LSTM-16f-128h")
    mlflow.set_tag("dev", "nick")

    model = ConvLSTMmodel(config=config)

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindata_loader=trainstreamer,
        validdata_loader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()

```

/Users/nickreinders/Documents/Master Applied Data Science deeltijd
HAN/Deployment/Raoul_repo/MADS-MachineLearning-course/.venv/lib/python3.11/site-
packages/torch/nn/modules/rnn.py:123: UserWarning: dropout option adds dropout
after all but last recurrent layer, so non-zero dropout expects num_layers
greater than 1, but got dropout=0.2 and num_layers=1

```
warnings.warn(
2025-09-28 16:02:08.563 | INFO      |
mltrainer.trainer:dir_add_timestamp:24 - Logging
to gestures/20250928-160208

2025-09-28 16:02:08.564 | INFO      |
mltrainer.trainer:__init__:68 - Found

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
100%|      | 81/81 [00:00<00:00, 141.75it/s]
2025-09-28 16:02:09.201 | INFO      |
mltrainer.trainer:report:209 - Epoch 0 train

2.7453 test 2.3562 metric ['0.1594']
100%|      | 81/81 [00:00<00:00, 128.88it/s]
2025-09-28 16:02:09.884 | INFO      |
mltrainer.trainer:report:209 - Epoch 1 train

2.2202 test 2.0002 metric ['0.2531']
100%|      | 81/81 [00:00<00:00, 159.72it/s]
2025-09-28 16:02:10.448 | INFO      |
mltrainer.trainer:report:209 - Epoch 2 train

1.9710 test 1.7906 metric ['0.3578']
100%|      | 3/3 [00:01<00:00, 1.60it/s]
```

```
[55]: trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 142.21it/s]
2025-09-28 16:02:42.708 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 3
2025-09-28 16:02:42.716 | INFO      |
mltrainer.trainer:report:209 - Epoch 3 train

1.8002 test 1.6284 metric ['0.4266']
100%|      | 81/81 [00:00<00:00, 141.46it/s]
2025-09-28 16:02:43.348 | INFO      |
mltrainer.trainer:report:209 - Epoch 4 train

1.4755 test 1.2189 metric ['0.5484']
100%|      | 81/81 [00:00<00:00, 147.65it/s]
2025-09-28 16:02:43.953 | INFO      |
mltrainer.trainer:report:209 - Epoch 5 train

1.1422 test 0.9109 metric ['0.6422']
100%|      | 3/3 [00:01<00:00, 1.61it/s]
```

```
[56]: trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 146.56it/s]
```

```

2025-09-28 16:03:03.637 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 6
2025-09-28 16:03:03.643 | INFO      |
mltrainer.trainer:report:209 - Epoch 6 train

0.8732 test 0.8268 metric ['0.6203']
100%|      | 81/81 [00:00<00:00, 156.70it/s]
2025-09-28 16:03:04.217 | INFO      |
mltrainer.trainer:report:209 - Epoch 7 train

0.8029 test 0.6676 metric ['0.7422']
100%|      | 81/81 [00:00<00:00, 157.49it/s]
2025-09-28 16:03:04.789 | INFO      |
mltrainer.trainer:report:209 - Epoch 8 train

0.6073 test 0.5434 metric ['0.7969']
100%|      | 3/3 [00:01<00:00, 1.71it/s]

```

```
[57]: trainer.loop()
```

```

100%|      | 81/81 [00:00<00:00, 141.07it/s]
2025-09-28 16:03:25.811 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 9
2025-09-28 16:03:25.817 | INFO      |
mltrainer.trainer:report:209 - Epoch 9 train

0.4980 test 0.4203 metric ['0.8516']
100%|      | 81/81 [00:00<00:00, 142.97it/s]
2025-09-28 16:03:26.448 | INFO      |
mltrainer.trainer:report:209 - Epoch 10 train

0.4559 test 0.3679 metric ['0.9000']
100%|      | 81/81 [00:00<00:00, 154.70it/s]
2025-09-28 16:03:27.028 | INFO      |
mltrainer.trainer:report:209 - Epoch 11 train

0.3238 test 0.2830 metric ['0.9250']
100%|      | 3/3 [00:01<00:00, 1.63it/s]

```

```
[58]: trainer.loop()
```

```

100%|      | 81/81 [00:00<00:00, 132.91it/s]
2025-09-28 16:03:36.990 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 12
2025-09-28 16:03:36.999 | INFO      |

```

```
mltrainer.trainer:report:209 - Epoch 12 train
0.2392 test 0.2432 metric ['0.9313']
100%|      | 81/81 [00:00<00:00, 119.90it/s]
2025-09-28 16:03:37.732 | INFO      |
mltrainer.trainer:report:209 - Epoch 13 train
0.2105 test 0.2453 metric ['0.9437']
2025-09-28 16:03:37.733 | INFO      |
mltrainer.trainer:__call__:252 - best loss:
0.2432, current loss 0.2453.Counter 1/5.
100%|      | 81/81 [00:00<00:00, 151.38it/s]
2025-09-28 16:03:38.334 | INFO      |
mltrainer.trainer:report:209 - Epoch 14 train
0.2009 test 0.2136 metric ['0.9344']
100%|      | 3/3 [00:02<00:00, 1.47it/s]
```